

Documentación técnica y guía de replicación

Asistente conversacional local para la exhibición de AudacIA: modelo de lenguaje, recuperación sobre el corpus institucional, voz y ventana de exhibición. Todo en la máquina, sin llamadas externas.

Índice

1	Presentación	4
	Arrancar en cinco minutos	4
	Qué tiene de particular	5
	Estructura	5
	Limitaciones conocidas	6
	Requisitos	6
2	Instalación desde cero	8
	1. Lo que hace falta antes de empezar	8
	2. Entorno de Python	8
	3. llama-cpp-python con CUDA	9
	3a. Rueda precompilada (empieza por aquí)	9
	3b. Compilar en la máquina	9
	4. El modelo	9
	5. Extras opcionales: ventana y voz	10
	6. Primer arranque y verificación	10
	7. Arrancar como se arranca el día de la exhibición	11
	8. Cuando algo falla	11
3	GitHub: publicar y replicar el proyecto	13
	Qué NO entra en el repositorio, y por qué	13
	A. Publicar el proyecto por primera vez	14
	A.1 Comprobar que no hay nada que no deba subir	14
	A.2 Inicializar y primer commit	14
	A.3 Crear el repositorio remoto	14
	A.4 Marcar la versión que se exhibió	15
	B. Replicar el proyecto en otra máquina	15
	C. Trabajar en el proyecto	16
	Ritmo de trabajo	16
	Mensajes de commit	17
	Cuando cambia el corpus	17
	Ganchos de pre-commit (opcional pero recomendado)	17
	Integración continua	17
	D. Problemas típicos	18
	E. Qué revisar antes de hacer público el repositorio	18
4	Docker: construir, ejecutar y replicar	19
	1. Antes de construir	19
	2. Qué hay dentro de la imagen	20
	3. Construir	20
	4. Ejecutar	20

Montar solo lo necesario	21
Con Compose	22
5. Verificar que el contenedor usa de verdad la GPU	22
Atención si tu GPU es de la serie RTX 50 (Blackwell)	22
6. Replicar el proyecto entero con Docker	23
7. Cuando algo falla	24
5 Arquitectura	26
Idea de fondo	26
El recorrido de un turno	26
Los módulos	27
Núcleo	27
Conocimiento	28
Memoria	28
Salida	28
Alrededor	28
Enrutado: tres dominios, uno heredado	29
Profundidad: la pregunta decide cuánto se trae	29
El presupuesto de contexto	30
Contaminación: la regla que más veces se ha roto	30
Concurrencia	31
Por dónde entrar según lo que quieras cambiar	31
6 Configuración	32
Quién manda sobre quién	32
Variables de entorno	32
Argumentos de run_hacu.py	33
Los bloques de config.py	33
ModelConfig — el modelo	33
RagConfig — el corpus y su recuperación	34
MemoryConfig — la memoria de los visitantes	34
VozConfig — oído y boca	35
InterfazConfig — la ventana	35
AppConfig — la raíz	35
El saludo de apertura	36
Cambiar el presupuesto de contexto sin romperlo	36
7 El corpus: lo que HACU sabe	37
Los dos niveles	37
El corpus no se escribe a mano	38
El tercer argumento, y por qué avisa	38
Después de cambiar el corpus	38
Escribir a mano, si hace falta	39
8 La capa de voz	40
Diagnóstico: lo primero que se corre al llegar a la sala	40
Reconocimiento	40
Síntesis	41
Cómo se escribe y cómo se dice	41

Callar a mitad de frase	42
Quién está hablando	42
Pulsar para hablar, y por qué es el modo por defecto	42
Solo altavoz	43
Los archivos	43
Limitaciones conocidas	43
9 La ventana de exhibición	44
Teclado	44
El panel del operador (F9)	45
Visitante	45
Audiencia	45
Voz	45
Memoria	45
Qué pasa al arrancar	46
Cómo se verifica sin mirarla	46
Los archivos	46
10 Pruebas	47
test_unidades — la que se corre siempre	47
recuperacion — ¿encuentra HACU lo que tiene documentado?	48
bateria — 60 entradas sueltas	48
conversacion — una visita con hilo	48
Cómo leer un fallo	49
Limpiar después	50
11 El día de la exhibición	51
La noche anterior	51
Al llegar a la sala, antes de levantar el modelo	51
Arrancar	52
Durante la jornada	52
Entre visitante y visitante	52
Ajustar al público	52
Cuando se pone pesado	52
Cuando dice algo raro	53
Si se queda colgado	53
Al cerrar	53
Chuleta	53
12 Datos de los visitantes	54
Qué se guarda	54
Qué NO se guarda	54
Qué filtra el sistema por su cuenta	54
Cómo se borra	55
Lo que hay que decidir antes de una exhibición real	55

Presentación

HACU (*Hardware de Audacia de Comunicación Universitaria*) es un asistente conversacional que responde sobre los proyectos de **AudacIA** y sobre la **Universidad Simón Bolívar** durante una exhibición abierta al público. Escucha por micrófono, responde en voz alta, recuerda a quien tiene delante durante la visita y admite cuando no sabe algo.

Corre **enteramente en local**: el modelo, el índice vectorial y la memoria de los visitantes no salen de la máquina. No hay llamadas a ninguna API.

```
python run_hacu.py --ui --voz --pantalla-completa
```

Arrancar en cinco minutos

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install llama-cpp-python --prefer-binary \
  --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cu121

# El modelo (~4,9 GB) no viene en el repositorio:
# -> models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf

pip install -r requirements-ui.txt      # ventana de exhibición (opcional)
pip install -r requirements-voz.txt    # micrófono y altavoz (opcional)
python -m hacu.voz --descargar          # voz de Piper (~110 MB)

python run_hacu.py --ui --voz
```

La primera ejecución descarga el modelo de embeddings (~470 MB) e indexa el corpus. Las siguientes arrancan directamente.

Paso a paso completo, con verificación y errores típicos: *Instalación desde cero*.

Comando	Qué hace
<code>python run_hacu.py</code>	Consola limpia, diagnóstico al log
<code>python run_hacu.py --debug</code>	Router, perfil activo, latencia y migraciones en pantalla
<code>python run_hacu.py --ui</code>	Ventana de exhibición, sin voz

Comando	Qué hace
<code>python run_hacu.py --voz-salida</code>	Consola: escribes tú, HACU responde en voz alta
<code>python run_hacu.py --ui --voz</code>	Ventana con micrófono y altavoz
<code>python run_hacu.py --ui --voz --pantalla-completa</code>	Lo que se proyecta el día de la exhibición
<code>python -m hacu.voz</code>	Diagnóstico de audio: qué micrófono y qué altavoz
<code>python -m pruebas.test_unidades</code>	383 comprobaciones, ~2 s, sin GPU
<code>python -m herramientas.limpiar</code>	Qué sobra en la carpeta de trabajo
<code>.\iniciar_robot.ps1</code>	Lo mismo dentro de Docker con GPU

Qué tiene de particular

Cuatro niveles de profundidad. Con 32 proyectos, «¿cuáles tienen?», «explícame cada uno» y «cuéntame todo sobre Mary» necesitan cosas distintas. La pregunta decide cuánto se recupera, y el índice-catálogo se carga entero en vez de buscarse — porque buscarlo devolvía 5 proyectos de 32.

No suelta el hilo. Un «¿y eso?» hereda el dominio del turno anterior, y la última respuesta viaja recortada como ancla. Una pregunta sobre HACU mismo no hereda: si no, una pregunta personal arrastraría documentación de un proyecto.

Capas deterministas después del modelo. Coletillas, adulación y fugas del andamiaje se recortan en Python, no se le piden al modelo. Lo que se lee, lo que se oye y lo que se guarda son exactamente el mismo texto.

Se calla cuando le dicen que se calle. El audio se entrega a la tarjeta en trozos de 50 ms: medido, 10 ms hasta detenerse. Quien interrumpe no espera a que termine la frase.

Nota cuándo se acerca otra persona, sin identificar a nadie. La referencia de timbre vive en memoria durante la visita y se borra al cambiar de visitante. No hay huella de voz en disco, y por tanto no hay base biométrica que custodiar.

Cada umbral sale de una medida. No hay un solo número en `config.py` que sea una suposición; al lado de cada uno está qué se midió para elegirlo.

Estructura

run_hacu.py	entrada
hacu/	
bootstrap.py	construye y cablea todo el sistema
cli.py	consola del operador
session.py	orquestación de un turno
identity.py	quién es el visitante; valida nombres, migra perfiles
routing.py	a qué corpus va la pregunta
context.py	qué se le manda al modelo
rag.py	indexación y recuperación sobre ChromaDB
prompts.py	system prompt, saludo y prompts de utilidad
llm.py	único acceso a llama.cpp, serializado
estilo.py	recorte determinista del cierre de turno
extractor.py	memoria episódica en segundo plano
sanitizer.py	qué merece guardarse como hecho
memory.py	persistencia SQLite
config.py	toda la configuración, con su justificación
voz/	micrófono, reconocimiento, síntesis, pronunciación
interfaz/	la ventana de exhibición
documents/	corpus en dos niveles (índice, fichas, institucional)
herramientas/	generación del corpus y limpieza del árbol
pruebas/	suite de regresión y herramientas de medición
docs/	esta documentación

Limitaciones conocidas

- El reconocimiento está medido de extremo a extremo con --autoprueba (4/4 nombres propios, ~1,9 s por frase en CPU). Lo que **no** se ha probado es la captura por micrófono real: nivel de entrada, ruido de sala y recorte de la primera sílaba solo se ven con hardware delante.
- Con altavoz abierto, la escucha automática hace que HACU se oiga a sí mismo. Pulsar-para-hablar lo evita; unos auriculares o un micrófono direccional también.
- El router es léxico. Las preguntas de seguimiento sin palabra clave se rescatan por distancia semántica, pero una referencia muy vaga puede traer el fragmento equivocado.
- Los nombres compuestos se limitan a dos palabras.
- Una sola estación: hay un perfil activo a la vez.
- La ruta de Docker no está verificada sobre la RTX 50 de esta máquina; la comprobación que falta está en 5. *Verificar que el contenedor usa de verdad la GPU.*

Requisitos

- GPU NVIDIA con CUDA y ≥ 8 GB de VRAM (desarrollado sobre una RTX 5070 Ti Laptop, 12 GB)
- Python 3.10 – 3.12
- ~12 GB de disco

- `models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf`

Corre sin GPU, pero en CPU un turno pasa de segundos a minutos: no sirve para una exhibición.

Instalación desde cero

Levantar HACU en una máquina que no lo ha visto nunca. Al final de esta página el sistema arranca, responde y se le puede hablar.

Si lo que buscas es **replicar el proyecto desde el repositorio**, empieza por *GitHub: publicar y replicar el proyecto* y vuelve aquí en el paso «Entorno de Python». Si prefieres el contenedor, ve a *Docker: construir, ejecutar y replicar*.

1. Lo que hace falta antes de empezar

	Mínimo	Con lo que está desarrollado
GPU	NVIDIA con ≥ 8 GB de VRAM	RTX 5070 Ti Laptop, 12 GB
Controlador	El que soporte tu CUDA	—
RAM	16 GB	32 GB
Disco	~12 GB libres	—
Python	3.10 – 3.12	3.11
Sistema	Windows 10/11 o Linux	Windows 11

HACU corre **sin GPU**, pero en CPU un turno pasa de segundos a minutos: no sirve para una exhibición. El arranque lo dice en la primera línea (`nvidia-smi`).

Comprueba el controlador antes de nada:

```
nvidia-smi
```

La esquina superior derecha de esa tabla dice la **versión de CUDA que soporta tu controlador**. Apúntala: decide qué rueda de llama-cpp-python instalar en el paso 3.

2. Entorno de Python

```
git clone <URL-DEL-REPOSITORIO> voz-rag-universidad
cd voz-rag-universidad

python -m venv venv
.\venv\Scripts\Activate.ps1          # Linux/macOS: source venv/bin/activate
```

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Si PowerShell se niega a ejecutar el script de activación:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

requirements.txt instala el núcleo: ChromaDB, los embeddings multilingües y el troceador de texto. **No** instala llama-cpp-python con CUDA — eso es el paso siguiente, porque depende de tu tarjeta.

3. llama-cpp-python con CUDA

Hay dos caminos. El primero tarda un minuto; el segundo, entre diez y cuarenta.

3a. Rueda precompilada (empieza por aquí)

```
pip install llama-cpp-python `
  --prefer-binary `
  --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cu121
```

Cambia cu121 por la serie que corresponda a tu CUDA (cu122, cu124, cu125...). **Si tu GPU es de la serie RTX 50 (Blackwell), cu121 puede no traer código nativo para tu arquitectura:** funcionará compilando sobre la marcha, o no funcionará en absoluto. Verifica con el paso 6 antes de darlo por bueno.

3b. Compilar en la máquina

Hace falta un compilador de C++ y CMake (en Windows, las *Build Tools for Visual Studio* con la carga de trabajo «Desarrollo para escritorio con C++»).

```
$env:CMAKE_ARGS="-DGGML_CUDA=on"
$env:FORCE_CMAKE="1"
pip install llama-cpp-python --no-cache-dir
```

```
CMAKE_ARGS="-DGGML_CUDA=on" FORCE_CMAKE=1 pip install llama-cpp-python --no-cache-dir
```

Compilar es más lento pero genera código para **tu** tarjeta, así que es la salida cuando ninguna rueda encaja.

4. El modelo

HACU espera este archivo exacto:

models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf

Son ~4,9 GB. **No está en el repositorio** y no debe estarlo (ver *Qué NO entra en el repositorio, y por qué*). Se descarga de Hugging Face, de cualquiera de los repositorios que publican la cuantización Q4_K_M de Meta-Llama-3.1-8B-Instruct:

```
pip install huggingface-hub
huggingface-cli download <repo-de-hugging-face> Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf \
  --local-dir models
```

El nombre del repositorio de Hugging Face depende de quién publique la cuantización y cambia con el tiempo; búscalo en <https://huggingface.co/models?search=Meta-Llama-3.1-8B-Instruct-GGUF> y comprueba que el archivo pesa ~4,9 GB. Llama 3.1 exige aceptar su licencia y `huggingface-cli login` antes de descargar.

Otro modelo GGUF sirve sin tocar código:

```
$env:HACU_MODELO="otro-modelo.gguf"      # relativo a models/, o ruta absoluta
$env:HACU_CTX="8192"                    # un modelo mayor deja menos VRAM
```

5. Extras opcionales: ventana y voz

Los dos son **opcionales a propósito**. El núcleo tiene que poder arrancar en una máquina sin tarjeta de sonido y sin entorno gráfico.

```
pip install -r requirements-ui.txt      # ventana de exhibición (PySide6)
pip install -r requirements-voz.txt    # micrófono, reconocimiento y síntesis
python -m hacu.voz --descargar          # voz de Piper, ~110 MB
```

En Linux, `sounddevice` necesita además `PortAudio` del sistema:

```
sudo apt install portaudio19-dev
```

Detalle completo de la capa de voz en *La capa de voz*.

6. Primer arranque y verificación

```
python run_hacu.py --debug
```

La primera vez descarga el modelo de embeddings (~470 MB) e indexa el corpus: tarda un par de minutos. Las siguientes arrancan directamente.

Lo que tiene que aparecer, en este orden:

```
--- DIAGNOSTICO DE HARDWARE ---
```

```
[OK] GPU detectada: ...
[*] Cargando el modelo en la VRAM...
[*] Sincronizando corpus institucional...
[*] Precalentando el modelo...
[OK] Sistema listo (precalentado en N.Ns).
```

--- HACU ---

Hola, soy Hacu. Bienvenido a AudacIA. ¿Serías tan amable de decirme cuál es tu nombre?

Verifica que el modelo está de verdad en la GPU, no solo que la GPU existe:

```
python -c "from llama_cpp import llama_supports_gpu_offload as g; print('GPU:', g())"
```

Si imprime False, llama-cpp-python se instaló sin CUDA: vuelve al paso 3 y usa 3b. Con el sistema arrancado, nvidia-smi debe mostrar ~5 GB ocupados y el precalentamiento debe tardar **segundos, no minutos**.

Lista de verificación completa:

```
python -m pruebas.test_unidades      # 383 comprobaciones, ~2 s, sin GPU
python -m pruebas.recuperacion       # calidad del RAG sobre el corpus real
python -m hacu.voz                   # inventario de micrófonos y altavoces
python -m hacu.voz --autoprueba      # circuito de voz completo sin micrófono
```

test_unidades en verde y recuperacion en 36/36 significan que el sistema está sano. Ver [Pruebas](#).

7. Arrancar como se arranca el día de la exhibición

```
python run_hacu.py --ui --voz --pantalla-completa
```

Comando	Qué hace
python run_hacu.py	Consola limpia, diagnóstico al log
python run_hacu.py --debug	Router, perfil, latencia y migraciones en pantalla
python run_hacu.py --ui	Ventana de exhibición, sin voz
python run_hacu.py --voz-salida	Consola: escribes tú, HACU responde en voz alta
python run_hacu.py --ui --voz	Ventana con micrófono y altavoz
python run_hacu.py --ui --voz --pantalla-completa	Lo que se proyecta
python -m hacu.voz	Diagnóstico de audio

8. Cuando algo falla

Síntoma	Causa	Qué hacer
No se detecta GPU NVIDIA	nvidia-smi no está en el PATH	Reinstalar el controlador
Arranca pero va lentísimo	llama-cpp-python sin CUDA	Paso 3b
ConfiguracionInviabile: El prompt estimado...	El presupuesto no cabe en n_ctx	Bajar HACU_FRAGMENTOS, o subir HACU_CTX
ConfiguracionInviabile: El embedding multilingue no esta disponible	Falta sentence-transformers o falló la descarga	Reinstalarlo; HACU_MULTILINGUE=0 acepta el modo degradado a sabiendas
Falta la interfaz grafica	Sin PySide6	pip install -r requirements-ui.txt
[mic] Oido: NO disponible	Sin micrófono o sin sounddevice	python -m hacu.voz; ver La capa de voz
Pausas larguísimas entre frases	Piper arranca un proceso por frase	El arranque lo avisa; python -m hacu.voz --descargar
CUDA out of memory al cargar Whisper	El STT no cabe junto al Llama	HACU_STT=tiny, o --voz-salida

El log completo queda en logs/hacu.log.

GitHub: publicar y replicar el proyecto

Cómo poner HACU en un repositorio y cómo alguien más lo baja y lo pone a funcionar. Pensado para que dentro de un año, con la exhibición montada otra vez, nadie tenga que reconstruir nada de memoria.

Qué NO entra en el repositorio, y por qué

Un repositorio de Git guarda **todas** las versiones de todo lo que entra. Un archivo de 5 GB añadido y borrado sigue pesando 5 GB en el historial, para siempre, para todo el que clone. GitHub además rechaza archivos de más de 100 MB.

Se queda fuera	Peso	De dónde sale
<code>models/*.gguf</code>	~4,9 GB	Se descarga de Hugging Face (4. El modelo)
<code>models/voz/</code> <code>chroma_db/</code>	~110 MB por voz ~decenas de MB	<code>python -m hacu.voz --descargar</code> Se reconstruye sola desde <code>documents/</code> al arrancar
<code>hacu_memory.db</code>	variable	Datos personales de visitantes. Nunca al repositorio
<code>logs/</code>	crece sin techo	Se abre uno nuevo en cada arranque
<code>pruebas/informes/</code>	~1 MB por corrida	Miden una versión concreta del corpus: envejecen mal
<code>venv/, __pycache__/, *.wav</code>	—	Basura de ejecución

Todo eso ya está en `.gitignore`. No lo quites.

Git LFS no es la solución para el modelo. LFS mueve el peso a otro almacén, pero sigue siendo peso, y la cuota gratuita de GitHub se mide en gigabytes de almacenamiento y de tráfico al mes: un puñado de clones de un `.gguf` de 4,9 GB la agota. El modelo es un artefacto público con URL propia: se descarga, no se versiona.

Lo que **sí** entra: el código, el corpus de `documents/` (son 60 KB de Markdown y son el conocimiento de HACU), las pruebas, las herramientas y esta documentación.

A. Publicar el proyecto por primera vez

A.1 Comprobar que no hay nada que no deba subir

```
cd D:\Proyectos\voz-rag-universidad

python -m herramientas.limpiar          # lista lo prescindible; con --aplicar lo borra
git status --short
git status --ignored --short             # confirma que el modelo y la memoria salen como
↪ ignorados
```

Antes del primer commit, revisa el tamaño de lo que va a entrar:

```
git add -A
git status --short | Measure-Object -Line
git ls-files -s | ForEach-Object { ($_ -split "\t")[1] } |
  ForEach-Object { Get-Item $_ } | Sort-Object Length -Descending |
  Select-Object -First 10 Length, Name
```

Si en esa lista aparece algo de más de un megabyte que no sea un .md del corpus, **para y averigua qué es** antes de seguir.

A.2 Inicializar y primer commit

Si el repositorio ya existe (hay carpeta .git), salta a A.3.

```
git init
git branch -M main
git add -A
git commit -m "HACU: asistente expositor de AudacIA"
```

.gitattributes fuerza finales de línea LF dentro del repositorio. Sin eso, un checkout en Windows escribe CRLF y el mismo árbol clonado en Linux o construido en el contenedor difiere del original, y git diff marca archivos enteros como cambiados sin que nadie los haya tocado.

A.3 Crear el repositorio remoto

Con la línea de órdenes de GitHub (gh, <https://cli.github.com>):

```
gh auth login
gh repo create voz-rag-universidad --private --source . --remote origin --push
```

O a mano: crear el repositorio **vacío** en <https://github.com/new> (sin README, sin .gitignore, sin licencia — los tiene ya el proyecto) y después:

```
git remote add origin https://github.com/<usuario>/voz-rag-universidad.git  
git push -u origin main
```

Privado o público. El corpus describe proyectos de investigación de la Universidad Simón Bolívar, con nombres de personas, patentes y aliados. Que eso se publique no es una decisión técnica: consúltalo con el centro antes de poner el repositorio en público. Con `--private` siempre se puede abrir después; al revés no.

A.4 Marcar la versión que se exhibió

Cuando una versión se lleva a una exhibición, ponle una etiqueta. Es lo que permite volver exactamente a lo que funcionó ese día:

```
git tag -a exhibicion-2026-09 -m "Versión llevada a la exhibición de septiembre"  
git push origin exhibicion-2026-09
```

B. Replicar el proyecto en otra máquina

Los pasos completos, desde un equipo que no tiene nada.

```
# 1. El código
git clone https://github.com/<usuario>/voz-rag-universidad.git
cd voz-rag-universidad

# 2. El entorno
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install llama-cpp-python --prefer-binary \
  --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cu121

# 3. El modelo (~4,9 GB; no viene en el repositorio)
#   Ver instalacion.md, paso 4
#   -> models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf

# 4. Los extras
pip install -r requirements-ui.txt
pip install -r requirements-voz.txt
python -m hacu.voz --descargar

# 5. Comprobar antes de confiar
python -m pruebas.test_unidades
python run_hacu.py --debug
```

El índice vectorial **no se clona y no hace falta clonarlo**: bootstrap.py lo reconstruye desde documents/ en el primer arranque, y en los siguientes solo sincroniza lo que haya cambiado. Un chroma_db/ copiado de otra máquina, con otra versión de ChromaDB, es una fuente de problemas y no ahorra nada.

Detalle de cada paso y qué hacer cuando falla: *Instalación desde cero*.

C. Trabajar en el proyecto

Ritmo de trabajo

```
git switch -c voz/callar-a-mitad-de-frase    # una rama por cambio
# ... trabajar ...
python -m pruebas.test_unidades             # NO se comitea en rojo
git add -A
git commit -m "La orden de llamar corta a mitad de frase, no en el siguiente punto"
git push -u origin voz/callar-a-mitad-de-frase
gh pr create --fill
```

python -m pruebas.test_unidades tarda dos segundos y no necesita GPU: no hay excusa para saltárselo. Las 383 comprobaciones existen porque cada una cubre algo que se rompió de verdad alguna vez.

Mensajes de commit

Describen **qué cambia de comportamiento**, no qué archivo se tocó. «Arreglos varios» no le sirve a nadie dentro de seis meses; «El índice se carga, no se busca: buscarlo devolvía 5 proyectos de 32» sí.

Cuando cambia el corpus

documents/ no se edita a mano: se regenera desde los informes en Word (ver *El corpus: lo que HACU sabe*). Después de regenerar, **medir siempre** antes de comitear:

```
python -m pruebas.recuperacion --n 6 --detalle
```

Ganchos de pre-commit (opcional pero recomendado)

Un gancho que corra la suite antes de cada commit evita el commit en rojo:

```
# .git/hooks/pre-commit (chmod +x; no se versiona: es local a cada clon)
#!/bin/sh
python -m pruebas.test_unidades || exit 1
```

Integración continua

pruebas/test_unidades.py corre sin GPU, sin red y sin tarjeta de sonido — está escrito así a propósito, y el bloque de la interfaz usa el backend offscreen de Qt. Eso lo hace apto para GitHub Actions tal cual. Guarda esto como .github/workflows/pruebas.yml:

```
name: pruebas
on: [push, pull_request]
jobs:
  unidades:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11"}
      - run: pip install langchain-text-splitters numpy PySide6
      - run: python -m pruebas.test_unidades
    env:
      QT_QPA_PLATFORM: offscreen
```

Nota: el flujo **no** instala requirements.txt, y es a propósito. La suite no necesita llama-cpp-python ni ChromaDB: los bloques que los usarían trabajan con dobles, y esas dos librerías se importan de forma perezosa. Las tres del pip install son las únicas que se importan a nivel de módulo en el camino de las pruebas. Verificado en un entorno limpio: **383/383 con PySide6, 365/365 sin él** (el bloque de la interfaz se omite solo). Si en el futuro un bloque nuevo importa el motor real, habrá que añadir la instalación y el flujo pasará de segundos a minutos.

D. Problemas típicos

Síntoma	Qué pasa	Solución
remote: error: File ... is 4.90 GB; this exceeds GitHub's file size limit	El modelo entró al commit	<code>git rm --cached models/*.gguf</code> , confirmar <code>.gitignore</code> , rehacer el commit
El modelo entró en un commit antiguo	Sigue en el historial aunque ya no esté en el árbol	Reescribir con <code>git filter-repo</code> , o empezar un repositorio limpio si aún no lo ha clonado nadie
<code>git diff</code> marca archivos enteros sin haberlos tocado	Finales de línea	<code>git add --renormalize .</code> una vez; <code>.gitattributes</code> evita que vuelva a pasar
Un clon nuevo no arranca	Falta el modelo, o las voces	Son descargas aparte: pasos 3 y 4 de la sección B
<code>hacu_memory.db</code> aparece en <code>git status</code>	Alguien lo añadió con <code>-f</code> , o el <code>.gitignore</code> se editó	<code>git rm --cached hacu_memory.db</code> y revisar quién tiene ya ese commit: son datos de personas

E. Qué revisar antes de hacer público el repositorio

1. `git log --all --stat | Select-String "hacu_memory|\.gguf|\.wav"` — que no haya datos de visitantes ni binarios pesados en **ningún** punto del historial.
2. El corpus de `documents/` lo aprueba el centro, no el repositorio.
3. `logs/` fuera: el log registra las preguntas de los visitantes.
4. Licencia: el proyecto no la declara todavía. Sin archivo `LICENSE`, un repositorio público sigue siendo «todos los derechos reservados» y nadie puede reutilizarlo legalmente — decidir con la universidad si eso es lo que se quiere.

Docker: construir, ejecutar y replicar

El contenedor sirve para **reproducir el entorno del modelo sin pelearse con CUDA en cada máquina**: la imagen trae la cadena de compilación, las dependencias y los embeddings ya descargados. Es la vía cómoda para levantar HACU en un servidor, en otra portátil o en la máquina de un compañero.

Lo que el contenedor NO da. La imagen instala solo `requirements.txt`: no lleva PySide6 ni la capa de voz, y un contenedor no tiene ni pantalla ni tarjeta de sonido del anfitrión. Dentro de Docker, HACU es **consola y texto**. `run_hacu.py --ui` responde [X] Falta la interfaz gráfica y termina. Para la exhibición —ventana, micrófono y altavoz— se usa la instalación nativa de *Instalación desde cero*. Esto no es una carencia que haya que arreglar: exponer audio y GPU a la vez desde un contenedor añade una capa de cosas que pueden fallar el día de la exhibición.

1. Antes de construir

	Windows	Linux
Motor	Docker Desktop con backend WSL 2	Docker Engine
GPU	Controlador NVIDIA con soporte WSL-CUDA	NVIDIA Container Toolkit
Disco	~15 GB para la imagen	ídem

En Linux, sin el Container Toolkit `--gpus all` no existe:

```
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Comprueba que Docker ve la tarjeta **antes** de construir nada:

```
docker run --rm --gpus all nvidia/cuda:12.1.1-base-ubuntu22.04 nvidia-smi
```

Si eso no imprime la tabla de `nvidia-smi`, el problema es del anfitrión y no de HACU. Arréglalo aquí; más adelante solo se vuelve más difícil de diagnosticar.

2. Qué hay dentro de la imagen

Dockerfile, de arriba abajo:

Capa	Qué hace	Por qué así
FROM nvidia/cuda:12.1.1-devel-ubuntu22.04	Base con CUDA y compilador	devel, no runtime: hace falta nvcc para llama.cpp
Python 3.10 + build-essential, cmake, git	Toolchain	Compilar llama.cpp si no hay rueda
COPY requirements.txt → pip install	Dependencias del núcleo	Capa propia, antes del código: cambiar una línea de Python no reinstala ChromaDB
Descarga de paraphrase-multilingual-MiniLM-L12-v2	Embeddings en /opt/hf-cache	Sin esto, cada docker run --rm sin volumen de caché bajaba ~470 MB
pip install llama-cpp-python --extra-index-url ../cu121	Motor del modelo	Se instala aparte de requirements.txt porque depende de CUDA
COPY . .	El código	Última capa: es lo que más cambia
CMD ["python", "run_hacu.py"]	Consola de HACU	—

.dockerignore deja fuera models/, chroma_db/, logs/, hacu_memory.db* y los informes de pruebas. La imagen pesa lo que pesan las dependencias, no lo que pesan los datos: el modelo se monta en tiempo de ejecución.

3. Construir

```
cd D:\Proyectos\voz-rag-universidad
docker build -t robot-expositor .
```

La primera construcción tarda: descarga la base de CUDA (varios GB), instala sentence-transformers con PyTorch y baja el modelo de embeddings. Las siguientes reutilizan las capas y, si solo cambió código Python, terminan en segundos.

4. Ejecutar

Lo más corto es el script que ya está en el proyecto:

```
.\iniciar_robot.ps1
```

Que es exactamente esto:

```
docker run --gpus all -it --rm `
-v "${PWD}:/app" `
robot-expositor
```

Parte	Para qué
--gpus all	Expone la GPU. Sin esto, HACU corre en CPU y un turno tarda minutos
-it	Terminal interactiva: HACU es una conversación, necesita entrada
--rm	Borra el contenedor al salir. El estado vive en el volumen, no en el contenedor
-v "\${PWD}:/app"	Monta la carpeta del proyecto completa

Ese único montaje hace todo el trabajo: dentro del contenedor, /app es la carpeta del anfitrión, así que el modelo de models/, el corpus de documents/, el índice chroma_db/ y la memoria hacu_memory.db son los mismos archivos que se ven en Windows. El índice que construye el contenedor queda en el disco del anfitrión y no hay que reconstruirlo en el siguiente arranque.

En Linux el script equivalente es:

```
docker run --gpus all -it --rm -v "$PWD:/app" robot-expositor
```

Con variables de configuración (*Configuración*):

```
docker run --gpus all -it --rm -v "${PWD}:/app" `
-e HACU_DEBUG=1 -e HACU_RETENCION=8 `
robot-expositor
```

Montar solo lo necesario

-v "\${PWD}:/app" monta **todo**, incluido el código: lo que corre es el código del anfitrión, no el que se copió en la imagen. Va muy bien mientras se desarrolla —se edita en Windows y se ejecuta dentro sin reconstruir— pero significa que la imagen no es autosuficiente. Para desplegar de verdad, monta solo los datos y deja que el código venga de la imagen:

```
docker run --gpus all -it --rm \
-v "$PWD/models:/app/models:ro" \
-v "$PWD/documents:/app/documents:ro" \
-v "$PWD/chroma_db:/app/chroma_db" \
-v "$PWD/logs:/app/logs" \
robot-expositor
```

hacu_memory.db queda deliberadamente fuera: sin él, cada arranque empieza sin memoria de visitantes, que es lo correcto en una demostración pública. Si la quieres persistente, añade -v

"\$PWD/hacu_memory.db:/app/hacu_memory.db" y lee *Datos de los visitantes* antes.

Con Compose

Si prefieres no recordar la orden larga, un `docker-compose.yml` como este hace lo mismo (no está en el repositorio; créalo si lo quieres):

```
services:
  hacu:
    build: .
    image: robot-expositor
    stdin_open: true
    tty: true
    volumes:
      - ./models:/app/models:ro
      - ./documents:/app/documents:ro
      - ./chroma_db:/app/chroma_db
      - ./logs:/app/logs
    environment:
      HACU_DEBUG: "1"
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
```

```
docker compose run --rm hacu
```

5. Verificar que el contenedor usa de verdad la GPU

Que `--gpus all` no proteste no significa que el modelo esté en la tarjeta.

```
docker run --rm --gpus all robot-expositor `
python -c "from llama_cpp import llama_supports_gpu_offload as g; print('GPU:', g())"
```

Tiene que imprimir `GPU: True`. Y con HACU arrancado, `nvidia-smi` en el anfitrión debe mostrar ~5 GB ocupados y el precalentamiento debe tardar segundos.

Atención si tu GPU es de la serie RTX 50 (Blackwell)

La imagen se construye sobre **CUDA 12.1**, y las ruedas que instala son de `cu121`. Esa versión de CUDA es **anterior** a la arquitectura de las RTX 50 (`sm_120`), así que el código nativo para tu tarjeta no está en la rueda. Puede salir adelante compilando PTX sobre la marcha en el primer arranque,

y puede no salir. **Compruébalo con la orden de arriba antes de confiar en el contenedor para una exhibición.**

Si sale False, o si el primer turno tarda minutos, sube las dos versiones —son dos líneas del Dockerfile:

```
FROM nvidia/cuda:12.8.1-devel-ubuntu22.04
...
RUN pip install --no-cache-dir llama-cpp-python --prefer-binary \
    --extra-index-url https://abetlen.github.io/llama-cpp-python/whl/cu128
```

Elige la serie (cu124, cu126, cu128...) que soporte tu controlador —la versión de CUDA que sale arriba a la derecha en `nvidia-smi`— y que exista en <https://abetlen.github.io/llama-cpp-python/whl/>. Si no hay rueda para esa combinación, quita el `--extra-index-url` y deja que compile: la imagen ya trae `cmake` y `build-essential`, y `CMAKE_ARGS="-DGGML_CUDA=on"` ya está puesto. La construcción pasa de minutos a media hora, pero genera código para tu tarjeta.

Esta ruta de Docker **no está verificada sobre la RTX 5070 Ti de esta máquina**: el desarrollo y todas las medidas del proyecto se hicieron sobre la instalación nativa de Windows. La comprobación de arriba es justo la que falta.

6. Replicar el proyecto entero con Docker

Desde una máquina que solo tiene Docker y el controlador NVIDIA:

```

# 1. El código
git clone https://github.com/<usuario>/voz-rag-universidad.git
cd voz-rag-universidad

# 2. El modelo (~4,9 GB; no viene en el repositorio ni en la imagen)
mkdir -p models
# Ver instalacion.md, paso 4
# -> models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf

# 3. La imagen
docker build -t robot-expositor .

# 4. Comprobar que la GPU está dentro
docker run --rm --gpus all robot-expositor \
  python -c "from llama_cpp import llama_supports_gpu_offload as g; print('GPU:', g())"

# 5. Comprobar que el sistema está sano (sin GPU, ~2 s)
docker run --rm robot-expositor python -m pruebas.test_unidades

# 6. Arrancar
docker run --gpus all -it --rm -v "$PWD:/app" robot-expositor

```

El paso 5 es barato y vale mucho: si la suite pasa dentro del contenedor, el enrutado, el estilo, la memoria y la profundidad de recuperación están intactos, y cualquier fallo posterior es del modelo o de la GPU, no del código. Serán **365 comprobaciones** y **no 383**: el bloque de la interfaz se omite solo, porque la imagen no lleva PySide6 a propósito.

7. Cuando algo falla

Síntoma	Causa	Solución
could not select device driver "" with capabilities: [[gpu]]	Falta NVIDIA Container Toolkit (Linux) o el backend WSL 2 (Windows)	Sección 1
manifest for nvidia/cuda:12.1.1-devel-ubuntu22.04 not found	Ese tag ya no se publica	Buscar uno vigente en https://hub.docker.com/r/nvidia/cuda/tags y cambiar el FROM
El arranque avisa No se detecto GPU NVIDIA	Falta --gpus all	Añadirlo
No such file or directory: '/app/models/Meta-Llama-...gguf'	El modelo no está montado (.dockerignore lo excluye a propósito)	Montar models/
[X] Falta la interfaz grafica	La imagen no lleva PySide6, y es correcto	La ventana va en nativo
Va lentísimo dentro y rápido fuera	El modelo está en CPU	Sección 5

Síntoma	Causa	Solución
La construcción tarda muchísimo en sentence-transformers	Está bajando PyTorch (~2 GB)	Normal la primera vez; después se cachea
chroma_db da errores raros tras usarlo desde Windows y desde el contenedor	Versiones distintas de ChromaDB sobre el mismo índice	<code>python -m herramientas.limpiar --aplicar --indice</code> y dejar que se reconstruya

El log del contenedor queda en `logs/hacu.log` del anfitrión si `logs/` está montado.

Arquitectura

Cómo está montado HACU por dentro, qué hace cada pieza y por qué está donde está.

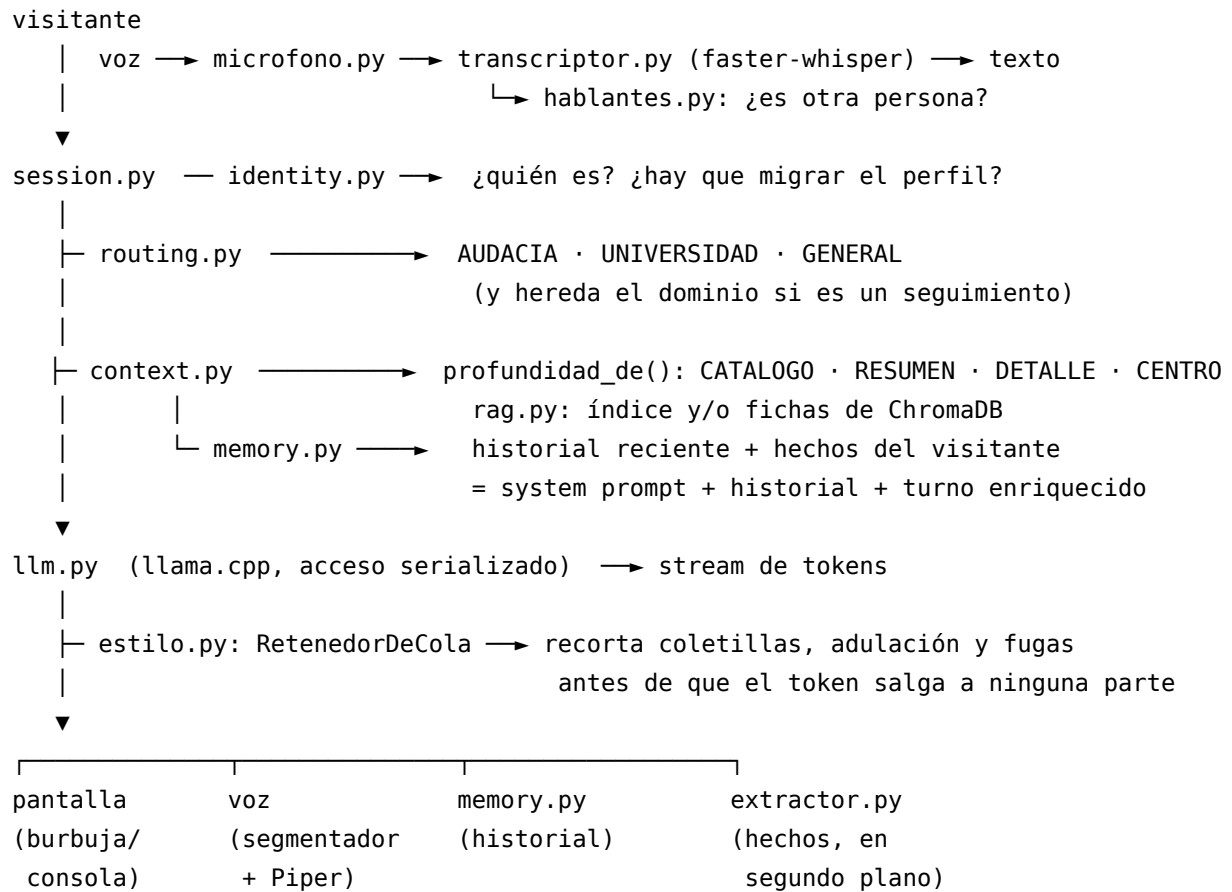
Idea de fondo

Todo corre **en local**: el modelo, el índice vectorial y la memoria de los visitantes no salen de la máquina. No hay llamadas a ninguna API. Eso condiciona cada decisión: la ventana de contexto es un presupuesto fijo, el modelo es un 8B cuantizado con manías conocidas, y no hay un modelo mayor esperando detrás para arreglar lo que salga mal.

De ahí salen las tres reglas que gobiernan el diseño:

1. **Lo determinista se hace en Python, no se le pide al modelo.** Recortar coletillas, filtrar adulación, trocear frases para la voz, decidir qué se recupera: todo eso tiene una implementación exacta y verificable. Al modelo se le pide solo lo que solo él puede hacer.
2. **Nada literal delante del modelo.** Una frase de ejemplo metida en un prompt acaba recitada palabra por palabra. Ha pasado cuatro veces en este proyecto (ver «Contaminación», abajo). Las reglas dicen *qué hacer*, no *qué decir*.
3. **Cada umbral sale de una medida.** No hay ningún número en `config.py` que sea una suposición; al lado de cada uno está qué se midió para elegirlo.

El recorrido de un turno



Lo que se **lee** en pantalla, lo que se **oye** y lo que se **guarda** son exactamente el mismo texto. Es la razón de que el filtrado de estilo ocurra en el retenedor y no en cada consumidor: si cada uno filtrara por su cuenta, acabarían divergiendo.

Los módulos

Núcleo

Archivo	Responsabilidad
run_hacu.py	Entrada: argumentos, diagnóstico de hardware, elige consola o ventana
hacu/bootstrap.py	<i>Composition root</i> . Construye y cablea todo el árbol una sola vez
hacu/config.py	Toda la configuración, con la justificación de cada valor
hacu/session.py	Orquestación de un turno. La consola, la ventana y las pruebas usan este mismo camino
hacu/llm.py	Único acceso a llama.cpp, serializado. Decodificación restringida a JSON para los prompts de utilidad

Archivo	Responsabilidad
hacu/prompts.py	System prompt, perfiles de audiencia, saludo de apertura y prompts de utilidad

Conocimiento

Archivo	Responsabilidad
hacu/routing.py	Enrutado léxico: a qué corpus va la pregunta. Aprende los nombres de proyecto del índice al arrancar
hacu/context.py	Qué se le manda al modelo: política de profundidad, anclaje de seguimientos, notas privadas
hacu/rag.py	Indexación por secciones ## y recuperación sobre ChromaDB

Memoria

Archivo	Responsabilidad
hacu/identity.py	Quién es el visitante: valida nombres, migra perfiles
hacu/memory.py	Persistencia en SQLite: historial y hechos
hacu/extractor.py	Extrae hechos de lo que dice el visitante, en un hilo aparte
hacu/sanitizer.py	Qué merece guardarse como hecho y qué no

Salida

Archivo	Responsabilidad
hacu/estilo.py	Recorte determinista: coletillas, adulación, fugas del andamiaje
hacu/cli.py	Consola del operador
hacu/interfaz/	Ventana de exhibición (<i>La ventana de exhibición</i>)
hacu/voz/	Oído y boca (<i>La capa de voz</i>)

Alrededor

Carpeta	Contenido
documents/	El corpus en dos niveles (<i>El corpus: lo que HACU sabe</i>)

Carpeta	Contenido
herramientas/	Generación del corpus desde los informes en Word, y limpieza del árbol
pruebas/	Suite de regresión y herramientas de medición (<i>Pruebas</i>)
docs/	Esta documentación

Enrutado: tres dominios, uno heredado

routing.py clasifica cada mensaje en AUDACIA, UNIVERSIDAD o GENERAL con patrones léxicos, no con un modelo: son microsegundos y es auditable.

Al arrancar, bootstrap.py le enseña al router **los nombres de los 32 proyectos**, leídos del índice-catálogo. Sin eso, «cuéntame de Holosand» caía en GENERAL y se saltaba toda la política de profundidad.

Un mensaje que el router no reconoce pero que se apoya en lo anterior —«¿y eso para qué sirve?», «cuéntame más», o cualquier frase que empiece por «y»— **hereda el dominio del último turno que sí lo tuvo**. Es más fiable que dejárselo a la distancia semántica, que no sabe de qué se venía hablando.

La excepción: las preguntas sobre HACU mismo («¿y a ti qué te gusta?») **no** heredan. Si heredaran, una pregunta personal arrastraría documentación de AudacIA y HACU contestaría con un proyecto.

Profundidad: la pregunta decide cuánto se trae

Con 32 proyectos, una sola forma de recuperar no sirve. context.py clasifica en cuatro niveles, y cada uno recupera algo distinto:

Nivel	Qué lo dispara	Qué recupera
CATALOGO	«¿qué proyectos tienen?»	el índice entero, cargado tal cual
RESUMEN	«explícame cada proyecto»	el índice como esqueleto + fichas
DETALLE	«cuéntame todo sobre Mary»	fichas, sin filtro
CENTRO	«¿qué patentes tiene el centro?»	solo lo institucional

Invariante: el índice se carga, no se busca. Queda excluido de toda búsqueda por similitud, por dos motivos medidos:

- Buscarlo devolvía 2 de sus 6 fragmentos, así que HACU enumeraba 5 proyectos de 32 y creía que esos eran todos.
- Al revés: siendo un documento genérico que menciona todo, ganaba consultas que no eran tuyas. A «¿cuántos estudiantes hay?» le llegó el catálogo y HACU respondió «32 proyectos» — un número de otra pregunta — en vez de admitir que no tenía el dato.

El nivel CENTRO existe por otra medida: el corpus de AudacIA son 37 fragmentos de fichas contra 10 institucionales, y sin filtrar, «¿qué patentes tiene el centro?» recuperaba cuatro proyectos de salud y ni una patente.

El presupuesto de contexto

La ventana son 16 384 tokens. `config.py` estima el peor caso —system prompt + fragmentos + hechos + historial— y `bootstrap.verificar_presupuesto()` **aborta el arranque** si no cabe.

No es exceso de celo. Cuando el contexto se desborda, `llama.cpp` descarta por la izquierda, y lo primero que se pierde es el system prompt: HACU cambiaría de personalidad a mitad de exhibición sin que nada lo indicara. Es mejor no arrancar.

```
python -c "from hacu.config import AppConfig; print(AppConfig().presupuesto_contexto())"
```

Actualmente: **10 274 de 15 484 tokens**.

Contaminación: la regla que más veces se ha roto

Un modelo de 8B copia cualquier cadena literal que se le ponga delante.

Ha ocurrido cuatro veces en este proyecto, siempre igual: un texto de andamiaje —un ejemplo dentro de un prompt, una nota de contexto, un saludo dictado— aparece literal en la respuesta al visitante.

Qué se puso delante	Qué recitó HACU
Un ejemplo con «Windows 11» en el prompt de extracción	«Windows 11» en la conversación
Una frase de perfil entera como ejemplo	La frase entera
La nota Recuperado de la documentacion institucional de la Universidad Simon Bolivar:	La frase, verbatim, en 11 de 30 turnos
La regla 10 decía «pregúntale qué le trae a la exhibición»	Esa pregunta exacta, ignorando lo que el visitante acababa de contar

Las defensas, en capas:

1. **Los prompts dicen qué hacer, no qué decir.** Las notas privadas se titulan «Lo que sabes de AudacIA», no con el nombre de ningún documento.
2. **El saludo de apertura no pasa por el modelo.** Es texto fijo que `session.saludar()` escribe directamente y registra en el historial.
3. **`pruebas/test_unidades.py` lo comprueba.** El bloque prompts verifica que el system prompt no nombra ningún proyecto del corpus; el bloque saludo, que el saludo no está dentro del system prompt.
4. **`estilo.limpiar_fugas()` borra en tiempo de ejecución lo que se cuele igual.**

Concurrencia

Hilo	Qué hace
Principal	Consola, o bucle de eventos de Qt
TrabajadorTurno	Generación del turno, fuera del hilo de Qt
TrabajadorTranscripcion	Reconocimiento de voz
TrabajadorEscuchaContinua	Escucha automática, cuando está activa
Locutor / TTS	Síntesis y reproducción, con su propia cola
BackgroundMemoryExtractor	Extracción de hechos, después del turno

l1m.py **serializa** todo acceso a llama.cpp con un cerrojo: el modelo es uno y no es reentrante, y el extractor de memoria compite con el turno del visitante por él. El cierre (bootstrap.cerrar) drena la cola del extractor antes de soltar la base de datos y la VRAM, para no perder hechos a medio escribir.

Por dónde entrar según lo que quieras cambiar

Quiero...	Empezar por
Cambiar cómo habla HACU	hacu/prompts.py, y después hacu/estilo.py
Añadir o corregir información	documents/, vía <i>El corpus: lo que HACU sabe</i>
Que reconozca un proyecto nuevo por su nombre	Nada: el router lo aprende del índice
Ajustar cuánto recupera	RagConfig en hacu/config.py; medir con pruebas/recuperacion.py
Cambiar la voz o la pronunciación	hacu/voz/pronunciacion.py, VozConfig
Tocar la ventana	hacu/interfaz/, verificable con python -m pruebas.test_unidades interfaz
Cambiar qué se recuerda de los visitantes	hacu/sanitizer.py y MemoryConfig; leer <i>Datos de los visitantes</i>

Configuración

Toda ruta, umbral y límite del proyecto vive en `hacu/config.py`, con la justificación de cada valor al lado. Ningún módulo construye rutas por su cuenta: se les inyecta la configuración. Las rutas son absolutas y derivadas de la raíz del repositorio, así que HACU arranca igual desde cualquier directorio de trabajo.

Quién manda sobre quién

argumentos de la línea de órdenes > variables de entorno > valores por defecto

`run_hacu.aplicar_argumentos()` aplica los argumentos **encima** de lo que ya resolvió `AppConfig.from_env()`. Así, `--sin-voz` gana aunque `HACU_VOZ=1` esté puesta en la sesión.

Variables de entorno

Lo que se puede cambiar sin tocar código ni reinstalar nada.

Variable	Efecto
<code>HACU_DEBUG=1</code>	Diagnóstico en consola: router, perfil, latencia, migraciones
<code>HACU_MODEL0=ruta.gguf</code>	Otro modelo (relativo a <code>models/</code> , o ruta absoluta)
<code>HACU_CTX=8192</code>	Ventana de contexto; un modelo mayor deja menos VRAM
<code>HACU_RETENCION=8</code>	Horas de historial que se conservan (0 desactiva la poda)
<code>HACU_FRAGMENTOS=4</code>	Fragmentos recuperados por consulta normal
<code>HACU_MULTILINGUE=0</code>	Acepta el embedding por defecto de Chroma; degrada el RAG a la mitad
<code>HACU_SALUDO="..."</code>	Otra frase de apertura. "" arranca sin saludar
<code>HACU_VOZ=1</code>	Activa micrófono y altavoz
<code>HACU_VOZ_SALIDA=1</code>	Solo altavoz: HACU habla pero no escucha
<code>HACU_VOZ_AUTO=1</code>	Escucha automática en vez de pulsar-para-hablar (solo en la ventana)
<code>HACU_HABLANTES=0</code>	No distinguir cuándo cambia la persona que habla
<code>HACU_STT=medium</code>	Tamaño del modelo de reconocimiento: tiny, base, small, medium

Variable	Efecto
HACU_TTS=sistema	Motor de síntesis: auto, piper, piper-proceso, piper-externo, sistema, mudo
HACU_PIPER=ruta.exe	Binario de Piper si no está en el PATH
HACU_VOZ_MODELO=es_ES-davefx-medium	Otra voz de Piper
HACU_ENTRADA=3	Índice del micrófono (ver python -m hacu.voz)
HACU_SALIDA=5	Índice del altavoz
HACU_PANTALLA_COMPLETA=1	La ventana arranca a pantalla completa

```
# PowerShell: valen para la sesión actual
$env:HACU_DEBUG="1"; $env:HACU_STT="tiny"
python run_hacu.py --ui --voz
```

```
HACU_DEBUG=1 HACU_STT=tiny python run_hacu.py --ui --voz
```

Argumentos de run_hacu.py

Argumento	Efecto
--debug	Como HACU_DEBUG=1
--ui	Ventana de exhibición en vez de consola
--voz	Micrófono y altavoz
--voz-salida	Solo altavoz
--sin-voz	Fuerza el modo escrito, ganando a las variables de entorno
--pantalla-completa	La ventana arranca a pantalla completa

Los bloques de config.py

ModelConfig — el modelo

Campo	Por defecto	Qué decide
model_path	models/Meta-Llama-3.1-8B-Instruct	Qué modelo se carga
n_ctx	16384	Ventana de contexto
n_gpu_layers	-1	Todas las capas a la GPU
chat_max_tokens	384	Techo de una respuesta normal. La regla 3 pide 2-4 frases y el modelo soltaba monólogos de nueve segundos

Campo	Por defecto	Qué decide
chat_max_tokens_extenso	900	Techo cuando se pide desarrollo. Enumerar 32 proyectos no cabe en 384
chat_temperature	0.3	Baja: en una exhibición importa más la fidelidad que la variedad
utility_*	—	Extracción y consolidación de memoria, con salida JSON restringida

RagConfig — el corpus y su recuperación

Campo	Por defecto	Qué decide
doc_folder	documents/	De dónde sale el conocimiento
db_path	chroma_db/	Dónde vive el índice vectorial
chunk_max_seccion	2200	Una ficha ## entra entera si cabe aquí
default_results	6	Fragmentos por consulta normal
broad_results_audacia / _universidad	6 / 8	Consultas amplias
fragmentos_resumen	4	Fichas que acompañan al índice en nivel RESUMEN
fragmentos_centro	6	Fragmentos institucionales en nivel CENTRO
multilingual_model	paraphrase-multilingual-MiniLM-L12-v2	El embedding
exigir_multilingue	True	Aborta el arranque si el multilingüe no está

Sobre esa última: con el embedding por defecto de Chroma —entrenado en inglés— la recuperación sobre el corpus en español baja de 36/36 a 27/36 consultas, y HACU afirmaría no conocer proyectos que sí están documentados. Para una exhibición eso es peor que no arrancar. HACU_MULTILINGUE=0 acepta el modo degradado a sabiendas.

MemoryConfig — la memoria de los visitantes

Campo	Por defecto	Qué decide
db_path	hacu_memory.db	La base SQLite. No se versiona
history_messages	10	Mensajes de historial que viajan al modelo (5 intercambios)
max_facts_per_profile	12	Hechos por visitante
consolidation_threshold	10	Cuándo se depuran los hechos
retencion_horas	24	Al arrancar se borra el historial más viejo. 0 desactiva

Campo	Por defecto	Qué decide
sanitize_on_startup	True	Limpia perfiles y hechos inválidos al arrancar

Ver *Datos de los visitantes*.

VozConfig — oído y boca

Detalle completo en *La capa de voz*. Lo que más se toca:

Campo	Por defecto	Qué decide
modelo_stt	small	~0,5 GB de VRAM; convive con el Llama de 8B en 12 GB
vocabulario	AudacIA, Holosand, Orion...	Nombres propios sembrados en el reconocedor
motor_tts	auto	Piper en proceso → Piper externo → voz del sistema → mudo
piper_voz	es_MX-claude-high	Voz descargable con <code>python -m hacu.voz --descargar</code>
pronunciaciones	()	Léxico propio; vacío usa el medido de <code>pronunciacion.py</code>
detectar_cambio_de_hablante	True	Nota cuándo se acerca otra persona, sin identificarla
umbral_hablante	0.65	Misma voz 0,844–0,947; voces distintas 0,314–0,448
minimo_frase / maximo_frase	12 / 240	Troceo del stream en frases pronunciables
deteccion_automatica	False	Escucha automática desde el arranque (solo ventana)

InterfazConfig — la ventana

Campo	Por defecto	Qué decide
pantalla_completa	False	Arrancar a pantalla completa
ancho / alto	1280 / 800	Tamaño de ventana
tamano_texto	17	Base del texto. En una tarima el público lee de lejos
mostrar_panel_operador	True	Panel derecho visible al arrancar

AppConfig — la raíz

Campo	Por defecto	Qué decide
default_user	visitante	Perfil anónimo
log_file	logs/hacu.log	Dónde va el log
debug_console	False	Diagnóstico en pantalla
saludo_inicial	«Hola, soy Hacu. Bienvenido a AudacIA. ¿Serías tan amable de decirme cuál es tu nombre?»	La primera frase. "" arranca callado

El saludo de apertura

Es **texto fijo**, no una respuesta del modelo: una frase literal puesta delante de un 8B es exactamente lo que acaba recitando en los turnos siguientes (ver *Contaminación: la regla que más veces se ha roto*).

Sí se registra en el historial como turno de HACU, y eso es lo importante: cuando el visitante conteste «Daniela», el modelo verá la pregunta justo encima. Sin eso, el primer mensaje de la conversación sería un nombre suelto sin nada que lo explique.

Tutea porque la regla 15 obliga a tutear sin cambiar de trato a mitad de conversación, y el saludo entra en el historial: un «usted» ahí arrastraría al modelo a usted durante el resto de la visita. Si se cambia con HACU_SALUDO, conviene mantener el tuteo por el mismo motivo.

En la ventana se repite al pulsar **Nuevo visitante**: quien acaba de acercarse tiene que oír la invitación a decir su nombre.

Cambiar el presupuesto de contexto sin romperlo

Subir HACU_FRAGMENTOS, history_messages o chat_max_tokens_extenso hace crecer el prompt. Si deja de caber en n_ctx, **el arranque se aborta** con ConfiguracionInviabile en vez de truncarse en escena.

```
python -c "from hacu.config import AppConfig; p,d = AppConfig().presupuesto_contexto();
↳ print(f'{p} de {d}')"

```

El corpus: lo que HACU sabe

Todo lo que HACU afirma sobre AudacIA y sobre la Universidad Simón Bolívar sale de `documents/`. Si algo no está ahí, HACU tiene que admitir que no lo sabe — y las pruebas comprueban que lo hace.

Los dos niveles

Con 32 proyectos, una sola forma de guardar la información no sirve para todas las preguntas. El corpus está en dos niveles:

Archivo	Nivel	Para qué
<code>audacia_indice_proyectos.md</code>	índice	los 32 proyectos a una línea; sirve para enumerar
<code>audacia_proyectos_salud.md</code>	fichas	una sección ## por proyecto, con su detalle
<code>audacia_proyectos_ambiente.md</code>	fichas	medio ambiente, agro e industria
<code>audacia_proyectos_cultura.md</code>	fichas	cultura, arte y turismo
<code>audacia_proyectos_hardware.md</code>	fichas	hardware, mecatrónica y robótica
<code>audacia_proyectos_educacion.md</code>	fichas	educación y EdTech
<code>audacia_proyectos_energia.md</code>	fichas	energía, servicios públicos y programas estratégicos
<code>audacia_proyectos_didacticos.md</code>	fichas	proyectos didácticos y experimentales
<code>audacia_centro.md</code>	institucional	personas, sedes, patentes, publicaciones, reconocimientos
<code>universidad_simon_bolivar.md</code>	institucional	historia, programas, ecosistema Eureka

El nombre del archivo decide el corpus: los que contienen «audacia» van a la colección de AudacIA; el resto, a la institucional. No hay configuración que mantener sincronizada.

La indexación respeta los encabezados ##: cada ficha entra **entera** en un fragmento y no se parte por la mitad. Cada fragmento lleva su tipo (`índice`, `ficha`, `institucional`), y eso es lo que permite pedirle al índice que enumere y a las fichas que expliquen. Ver *Profundidad: la pregunta decide cuánto se trae*.

El corpus no se escribe a mano

Se genera desde los informes en Word, de modo que actualizar la documentación no obliga a transcribir nada y no se pueden colar datos inventados.

```
# 1. Extraer el texto de los .docx a un JSON intermedio
python herramientas/extraer.py <carpeta con los .docx> fuente.json

# 2. Construir el corpus en dos niveles
python herramientas/construir_corpus.py fuente.json documents/ [corpus_anterior.md]

# 3. MEDIR. Siempre. Antes de comitear nada.
python -m pruebas.recuperacion --n 6 --detalle
```

herramientas/breves.py es **lo único escrito a mano**: el resumen de una línea de cada proyecto para el índice-catálogo. Es compresión editorial de las descripciones del propio informe, y existe porque el índice tiene que caber entero en el contexto para poder enumerar los 32 proyectos de una vez.

El tercer argumento, y por qué avisa

El detalle técnico de los **proyectos didácticos** nunca estuvo en los informes en Word: se escribió a mano en el corpus anterior (documents/audacia_unisimón.md, hoy borrado del árbol). Sin ese archivo el corpus se regenera igual, pero las fichas didácticas salen más pobres que las que hay ahora en disco, y el fallo es silencioso: los archivos aparecen, solo que con menos dentro. Por eso el generador **avisa a gritos** cuando falta.

Si el archivo estuvo alguna vez en el repositorio, se recupera sin resucitarlo:

```
git log --oneline --all -- "documents/audacia_unisimón.md"
git show <commit>:"documents/audacia_unisimón.md" > audacia_anterior.md
python herramientas/construir_corpus.py fuente.json documents/ audacia_anterior.md
```

Después de cambiar el corpus

1. **Reindexar** — no hay que hacer nada: bootstrap.py sincroniza al arrancar. Los fragmentos de documentos que ya no están en la carpeta se eliminan del índice, así que borrar un archivo lo borra también del conocimiento de HACU.
2. **Medir la recuperación:**

```
python -m pruebas.recuperacion --n 6 --detalle
```

La referencia actual es 36/36 con el embedding multilingüe (27/36 con el de Chroma por defecto). Una consulta que baja es un proyecto que HACU dejará de encontrar.

3. **Comprobar que el índice sigue completo:** pruebas/recuperacion.py verifica que el índice-catálogo entrega los 32 proyectos. Si se añade uno al informe y no a breves.py, esa comprobación lo caza.
4. **Correr el guion:**

```
python -m pruebas.conversacion
```

Varios turnos citan datos concretos del corpus; si un dato se movió de sitio, se ve aquí.

Si el índice se queda en un estado raro (por ejemplo tras usar el mismo chroma_db desde Windows y desde el contenedor):

```
python -m herramientas.limpiar --aplicar --indice
```

Se reconstruye entero en el siguiente arranque, un par de minutos.

Escribir a mano, si hace falta

Nada impide añadir un .md directamente a documents/. Las reglas:

- **El nombre decide el corpus:** con «audacia» dentro va a Audacia; si no, a la colección institucional.
- **Una sección ## por unidad de conocimiento.** Es la unidad de recuperación: todo lo que tiene que llegar junto al modelo va bajo el mismo encabezado.
- **Ninguna sección de más de ~2 200 caracteres** (chunk_max_seccion) si se quiere que entre entera en un fragmento.
- **Sin frases de andamiaje.** Lo que se escriba en el corpus puede acabar recitado literalmente por el modelo.
- Después: medir con pruebas/recuperacion.py.

Lo que se añada a mano se pierde en la siguiente regeneración desde los .docx. Si un dato tiene que sobrevivir, tiene que estar en el informe o en breves.py.

La capa de voz

Oído y boca de HACU. Toda la capa es **opcional**: si falta una dependencia o no hay tarjeta de sonido, ServicioDeVoz.disponible es False y HACU sigue funcionando por escrito. Una exhibición sin sonido es mala; una exhibición caída es peor.

Diagnóstico: lo primero que se corre al llegar a la sala

```
python -m hacu.voz                # lista micrófonos y altavoces con su índice
python -m hacu.voz --probar        # graba 3 s y los reproduce
python -m hacu.voz --hablar "Hola, soy Hacu"
python -m hacu.voz --descargar    # baja la voz de Piper configurada
python -m hacu.voz --calibrar     # mide el ruido de la sala

# Sin depender del altavoz ni del micrófono de la máquina
python -m hacu.voz --hablar "Hola, soy Hacu" --guardar prueba.wav
python -m hacu.voz --transcribir grabacion.wav
python -m hacu.voz --autoprueba   # circuito completo sin micrófono
```

Se corre **antes de levantar el modelo**. En una portátil con webcam, base de conexiones y auriculares llega a haber cinco entradas de audio, y el sistema no siempre elige la que uno cree. Los índices se fijan con HACU_ENTRADA y HACU_SALIDA.

--guardar vuelca la síntesis a un WAV sin reproducirla y --transcribir hace el camino inverso. Sirven para separar dos fallos que suenan igual: que HACU corte la frase, o que la corte la salida de audio. **Si el WAV está entero, el problema no es de HACU.**

--autoprueba cierra el circuito sin hardware: sintetiza cuatro frases de exhibición con Piper y se las transcribe a sí mismo, comprobando que los nombres propios sobreviven al reconocedor. Es la única forma de saber si el reconocimiento funciona en una máquina donde todavía no hay micrófono.

Reconocimiento

faster-whisper small en int8, ~0,5 GB de VRAM: convive con el Llama de 8B dentro de los 12 GB de la portátil.

Los nombres propios de la exhibición van sembrados en el prompt inicial del reconocedor (vocabulario en VozConfig), y eso no es una precaución teórica. Medido con --autoprueba, sobre las mismas frases:

Frase	Con vocabulario	Sin vocabulario
«¿Qué proyectos tiene AudacIA?»	AudacIA	«a UDAC ya»
«...sobre Holosand y el sensor Kinect»	Holosand, Kinect	«olo San», «Kinex»

Con el nombre destrozado, el router no reconoce el dominio y HACU responde que no sabe de un proyecto que sí está documentado.

Si el reconocedor no cabe junto al modelo (CUDA out of memory), HACU_STT=tiny baja el consumo, y --voz-salida lo quita del todo.

Síntesis

Piper si está instalado; si no, la voz del sistema operativo (SAPI5 en Windows). El respaldo suena a robot de los noventa, pero existe en cualquier máquina y hace que HACU hable el primer día sin descargar nada.

HACU **no espera a terminar de generar para hablar**: un segmentador trocea el stream en frases y pronuncia la primera mientras el modelo escribe la segunda.

Piper se carga **dentro del proceso**, una sola vez al arrancar. El respaldo que lanza `python -m piper` por frase sigue ahí, pero solo como último recurso: medido, **0,21 s por frase en proceso frente a 2,52 s lanzando el proceso**, y en una máquina Windows con una voz high esa diferencia sube a más de diez segundos de silencio en cada punto. El arranque dice qué motor se eligió y avisa si acabó en el lento.

HACU_TTS acepta `piper-proceso`, `piper-externo`, `sistema` o `mudo` para forzar uno.

Cómo se escribe y cómo se dice

`hacu/voz/pronunciacion.py` reescribe el texto **solo para la voz**, en el último paso antes de sintetizar: la pantalla y la memoria conservan la ortografía real. El visitante ve «AudacIA» y oye «Audacia».

Hace falta porque la ortografía de la exhibición engaña al sintetizador. Medido sintetizando la frase y transcribiendo lo sintetizado, que es la única forma de saber qué sale por el altavoz sin tener a alguien escuchando:

Escrito	Se oía	Se oye ahora
Hacu	«Aku»	«Jacu»
AudacIA	«Uga Ia», «Oda ya»	«Audacia»
IoT	«yote»	«Internet de las Cosas»
3.000 m ²	«temedos»	«metros cuadrados»
OEA	«olla», «Bahía»	«Organización de los Estados Americanos»
SkinnIA	«Skinny a»	«Skinia»

Las siglas que el castellano ya deletrea bien (ROV, ROP, PCR, HLB) **no** están en el léxico: no hay reglas de más. «OEA» se dice por su nombre completo porque Piper no la pronuncia en ninguna grafía — se probó OEA, O.E.A. y O E A, y las tres salían mal.

El léxico se amplía sin tocar código con `VozConfig.pronunciaciones`. Si se define, **sustituye** al léxico medido: hay que incluir también las entradas que se quieran conservar.

Callar a mitad de frase

Callar a HACU **corta de inmediato**, no en el siguiente punto: el audio se entrega a la tarjeta en trozos de 50 ms en vez de frase entera. Medido: **10 ms hasta detenerse y 36 ms de audio ya en vuelo**.

Vale para el botón «Callar a HACU», para Esc, y para cuando el visitante vuelve a pulsar la barra espaciadora para hablar — que es lo importante: quien interrumpe espera que el robot se calle, no que termine la frase.

Quién está hablando

HACU nota cuando **se acerca otra persona** y cierra el perfil anterior, para que el siguiente visitante no herede el nombre ni los hechos del que acaba de irse.

Lo que hace y lo que deliberadamente no hace:

- **No identifica a nadie.** Solo dice «esta es otra voz», nunca «esta es Camila».
- **No guarda nada.** La referencia de timbre vive en memoria durante la visita y se borra al pasar al siguiente visitante o al cerrar. No hay huella de voz en disco, y por tanto no hay base biométrica que custodiar.

La distinción importa: un vector de voz almacenado es un dato biométrico, y esto es una exhibición abierta al público con menores entre el público. Guardarlo sería una decisión de la universidad, no de un commit. Ver *Datos de los visitantes*.

Calibrado con dos voces distintas sintetizadas: **misma voz 0,844–0,947, voces distintas 0,314–0,448**, y el umbral por defecto (0,65) cae en medio de ese hueco. Con voces humanas y ruido de sala la separación será menor, así que cada comparación queda en el log para poder recalibrar en la sala. `HACU_HABLANTES=0` lo desactiva.

Pulsar para hablar, y por qué es el modo por defecto

En una tarima el altavoz alimenta al micrófono: con detección automática HACU se oye a sí mismo y se responde solo. No es comodidad, es que el otro modo no funciona en una sala abierta.

La escucha automática está implementada y se activa desde el panel de la ventana —calibra el ruido de sala al encenderla—, pero es para sala controlada o auriculares. `HACU_VOZ_AUTO=1` la activa desde el arranque, y espera a que HACU termine de saludar antes de calibrar: medir el ruido de sala mientras el altavoz está sonando dejaría el umbral por encima de cualquier voz humana.

En consola, la escucha automática no está disponible: se habla con 9 o v, que abren el micrófono hasta el siguiente Enter.

Solo altavoz

--voz-salida monta la boca y no el oído: no descarga el modelo de reconocimiento ni necesita entrada de audio. Sirve para probar la voz desde una sesión remota, y también el día que el micrófono falle a media jornada — HACU sigue hablando y el operador escribe las preguntas.

Los archivos

Archivo	Responsabilidad
microfono.py	Captura con sounddevice: pulsar-para-hablar y escucha hasta silencio
deteccion.py	Detección de voz por energía. Lógica pura, verificable sin hardware
transcriptor.py	faster-whisper, con el vocabulario sembrado
sintetizador.py	Piper en proceso, Piper externo, voz del sistema y modo mudo
segmentador.py	Trocea el stream de tokens en frases pronunciables
pronunciacion.py	Léxico de «cómo se escribe / cómo se dice»
hablantes.py	Cambio de timbre, efímero y sin identificar
dispositivos.py	Inventario y diagnóstico de audio
__init__.py	ServicioDeVoz: los cuatro verbos que ven la consola y la ventana
__main__.py	El diagnóstico de <code>python -m hacu.voz</code>

Limitaciones conocidas

- El reconocimiento está medido de extremo a extremo con --autopruueba (4/4 nombres propios, ~1,9 s por frase en CPU). Lo que **no** se ha probado es la captura por micrófono real: el nivel de entrada, el ruido de sala y el recorte de la primera sílaba solo se ven con hardware delante.
- Con altavoz abierto, la escucha automática hace que HACU se oiga a sí mismo.
- Una sola estación: hay un perfil activo a la vez.

La ventana de exhibición

```
python run_hacu.py --ui --voz --pantalla-completa
```

Tres zonas, cada una para un público distinto.

Izquierda — el núcleo animado, el botón de hablar y el medidor de nivel. Es lo que ve el visitante desde lejos y lo único que necesita entender.

Centro — la conversación en burbujas, que crece token a token mientras el modelo escribe.

Derecha — los mandos del operador, que se ocultan con F9 para que el público no vea la cocina.

El núcleo cambia de color según el estado: **gris** en espera, **verde** escuchando (los anillos siguen el nivel real del micrófono), **ámbar** pensando, **turquesa** hablando. Es el único indicador que se lee desde el fondo de una sala.

Teclado

Tecla	Efecto
Espacio (mantener)	Hablarle a HACU
Enter en el recuadro	Enviar lo escrito
F9	Ocultar o muestra el panel del operador
F11	Pantalla completa
Esc	Callar a HACU en mitad de una frase, y recuperar el teclado
Ctrl+Q	Cerrar (a pantalla completa no hay barra de título)

El teclado se queda en la ventana, no en el recuadro de texto. El recuadro solo lo toma si haces clic en él, y lo suelta al enviar. De lo contrario la barra espaciadora escribía un espacio en vez de abrir el micrófono, y había que hacer clic fuera del recuadro para poder hablarle.

Ctrl+C en la terminal también cierra la ventana, y cierra **bien**: el bucle de eventos de Qt vive en C++ y no devuelve el control al intérprete, así que la señal necesita un latido que le abra hueco y un manejador que pase por el cierre normal, en vez de matar el proceso con el modelo y la base a medias.

Si no hay micrófono, el botón se deshabilita solo y queda el campo de texto: la ventana funciona igual.

El panel del operador (F9)

Visitante

Mando	Qué hace
Etiqueta de perfil	Nombre activo y cuántos hechos recuerda de él. Se refresca sola cada 2 s, porque el extractor de memoria trabaja en segundo plano y termina después del turno
Fijar nombre a mano	Para cuando el reconocimiento destroza un nombre
Nuevo visitante	Cierra el perfil, limpia la pantalla y vuelve a saludar. Es el botón entre visita y visita

Audiencia

Mando	Qué hace
Desplegable	General, Técnico, Infantil o Artístico. Cambia el registro, no el contenido
Modo trivia	HACU propone preguntas sobre la universidad y los proyectos, una a una

Voz

Mando	Qué hace
Escucha automática	Calibra el ruido de sala y escucha sin pulsar nada. Con altavoz abierto HACU se oye a sí mismo: es para sala controlada o auriculares
Callar a HACU	Corta la frase en curso de inmediato (≈ 10 ms). Igual que Esc

Memoria

Mando	Qué hace
Ver lo que recuerda	Los hechos del perfil activo, tal cual están guardados
Limpiar la pantalla	Borra las burbujas y el chat inmediato; conserva los hechos
Borrar TODO	Purga la base entera, con confirmación. No se puede deshacer

Qué pasa al arrancar

1. Se monta la ventana y se aplican los estilos.
2. Se anota cualquier problema de la capa de voz (sin micrófono, motor lento...).
3. Se recuerdan las dos cosas que el operador necesita: la barra espaciadora y `Ctrl+Q`.
4. **HACU saluda**: aparece su primera burbuja y, si hay altavoz, la dice en voz alta.

El saludo es texto fijo, no una respuesta del modelo, pero se registra como turno suyo para que la respuesta del visitante llegue con la pregunta delante. Ver *El saludo de apertura*.

Cómo se verifica sin mirarla

El bloque interfaz de la suite monta la ventana de verdad contra un modelo falso, en el backend offscreen de Qt: corre en cualquier máquina, sin tarjeta gráfica y sin GPU.

```
python -m pruebas.test_unidades interfaz
```

No comprueba que sea bonita —eso se mira— sino que se monta, que el turno llega al hilo correcto, que el texto que se pinta es el mismo que se habla, que los atajos existen, que el foco del teclado se comporta y que la ventana abre saludando.

Los archivos

Archivo	Responsabilidad
ventana.py	La ventana: montaje, estado, turnos, mandos
widgets.py	Núcleo animado, medidor de nivel, botón de hablar, burbujas, métricas
hilos.py	Turnos, transcripción y tareas largas fuera del hilo de Qt
estilos.py	Paleta, hoja de estilos y rótulos de estado

La ventana no sabe nada de `llama.cpp`, de ChromaDB ni de PortAudio: habla con `HacuSession` y con `ServicioDeVoz`, y todo lo lento va a un hilo.

Pruebas

Cinco herramientas, cada una para una pregunta distinta.

Orden	Qué mide	Cuánto tarda	¿GPU?
<code>python -m pruebas.test_unidades</code>	Regresión completa de las capas deterministas y el turno	~2 s	no
<code>python -m pruebas.recuperacion</code>	Calidad del RAG sobre el corpus real	~20 s	no
<code>python -m pruebas.bateria --seco</code>	60 entradas sueltas contra las capas deterministas	~5 s	no
<code>python -m pruebas.bateria --vivo</code>	Las mismas 60 contra el modelo real	minutos	sí
<code>python -m pruebas.conversacion</code>	Una visita completa de 30 turnos encadenados	minutos	sí

Después de cada cambio: `test_unidades`. **Antes de una exhibición:** `bateria --vivo` y `conversacion`.

test_unidades — la que se corre siempre

383 comprobaciones, sin GPU, sin red, sin tarjeta de sonido y sin dependencias extra. Cada una existe porque algo se rompió de verdad alguna vez.

```
python -m pruebas.test_unidades          # todo
python -m pruebas.test_unidades estilo   # solo los bloques que casen
python -m pruebas.test_unidades voz saludo
```

Los bloques: routing, seguimiento, nombres, identity, sanitizer, estilo, calidez, truncado, memory, context, profundidad, session, saludo, prompts, guion, cierre, voz, hablantes, pronunciacion, interfaz, config.

El bloque interfaz monta la ventana de verdad en el backend offscreen de Qt, así que corre igual en una máquina sin pantalla. El bloque prompts comprueba la invariante de contaminación: que el system prompt no nombre ningún proyecto del corpus.

Tarda dos segundos y no necesita GPU. **No se comitea en rojo.**

recuperacion — ¿encuentra HACU lo que tiene documentado?

Mide el RAG solo, sin el modelo: 36 consultas contra el corpus real, comprobando que cada una recupera el fragmento que le corresponde.

```
python -m pruebas.recuperacion --n 6 --detalle
```

Referencia actual: **36/36 con el embedding multilingüe, 27/36 con el de Chroma por defecto**. Esa diferencia es la razón de que el arranque se aborte si el multilingüe no está disponible.

También comprueba que el índice-catálogo entrega **los 32 proyectos**: si se añade uno al informe y no a herramientas/breves.py, se ve aquí.

Es la prueba que se corre **después de tocar el corpus** (*El corpus: lo que HACU sabe*).

bateria — 60 entradas sueltas

Mide el turno, no la conversación: 60 entradas barajadas, cada una con su intención esperada, su efecto sobre el perfil y su criterio de aceptación.

```
python -m pruebas.bateria --seco           # capas deterministas
python -m pruebas.bateria --vivo          # contra el modelo real
python -m pruebas.bateria --listar        # imprime el catálogo sin ejecutarlo
python -m pruebas.bateria --semilla 7     # fija el orden
```

--vivo usa una base de datos temporal y deja un informe en JSON y CSV en pruebas/informes/. Esos informes **no se versionan**: miden una versión concreta del corpus y envejecen mal.

PRUEBAS.md describe las 60 entradas y su criterio. Se regenera:

```
python -m pruebas.bateria --listar > PRUEBAS.md
```

conversacion — una visita con hilo

La batería mide entradas sueltas y barajadas. Esto mide lo contrario: **una sola visita**, en orden fijo, con un mismo visitante, donde cada turno depende del anterior. Es la prueba que se parece a lo que va a pasar en la tarima.

```
python -m pruebas.conversacion           # los 30 turnos
python -m pruebas.conversacion --listar   # imprime el guion sin ejecutarlo
python -m pruebas.conversacion --desde G15 # arranca en un turno concreto
python -m pruebas.conversacion --tipo CORRECCION
python -m pruebas.conversacion --voz      # además, hablando
```

El guion (pruebas/guion.py) tiene seis tipos de turno:

Tipo	Qué mide
ESPECIFICA	dato puntual documentado; la respuesta debe ser corta
EXTENDIDA	pide desarrollo; la respuesta debe pasar de ~500 caracteres
SEGUIMIENTO	«¿y eso para qué sirve?»: no se sostiene sin el turno anterior
CORRECCION	el visitante afirma algo falso y HACU tiene el dato correcto
SIN_DATO	no está en el corpus; hay que admitirlo, incluso bajo presión
PERSONAL	acoge el nombre, lo recuerda, no revela de dónde lo sabe

Cada turno declara qué términos del corpus **debe** contener la respuesta y qué invenciones **no** puede contener, con comprobación sensible a la negación: «Holosand no usa gafas» no cuenta como haber dicho «gafas».

Los turnos G17 y G18 reproducen la alucinación del observatorio astronómico y el desafío de fuentes que la siguió, de modo que esa regresión queda cubierta por una comprobación y no por la memoria de nadie. Los turnos G26 a G30 recorren los niveles de profundidad: contar los proyectos, enumerar un área entera y dar la cátedra de uno solo sin soltar el hilo.

GUION.md es la versión legible del guion. Se regenera:

```
python -m pruebas.conversacion --listar > GUION.md
```

Cómo leer un fallo

Un fallo de test_unidades es **siempre** un fallo del código: no hay modelo detrás, todo es determinista.

Un fallo de bateria --vivo o de conversacion puede ser del código o del modelo. Antes de tocar nada, repetir ese turno solo:

```
python -m pruebas.conversacion --desde G17
```

Si el fallo no se repite, era variabilidad del modelo. Si se repite, es real.

Y hay una tercera posibilidad, que ha pasado varias veces: **el criterio de la prueba estaba mal**. Un turno que exige la palabra «cultivo» cuando el corpus dice «agricultura» falla sin que nada esté roto. Antes de cambiar el sistema, leer la respuesta completa y preguntarse si de verdad está mal.

Limpiar después

```
python -m herramientas.limpiar          # lista lo prescindible  
python -m herramientas.limpiar --aplicar # lo borra
```

Los informes de pruebas, el log y el bytecode se acumulan entre corridas. La base de memoria y el índice vectorial van aparte (--memoria, --indice) porque uno son datos de personas y el otro cuesta un par de minutos de reconstrucción.

El día de la exhibición

Guion de operación: qué hacer antes de abrir, durante la jornada y al cerrar.

La noche anterior

```
cd D:\Proyectos\voz-rag-universidad
.\venv\Scripts\Activate.ps1

python -m pruebas.test_unidades      # 383 en verde
python -m pruebas.recuperacion       # 36/36
python -m pruebas.conversacion       # la visita completa, contra el modelo real
```

Si la conversación falla en algún turno, hay tiempo de mirarlo. A las nueve de la mañana con público delante, no.

Comprobar que el modelo está donde se espera y que la GPU lo acepta:

```
python -c "from llama_cpp import llama_supports_gpu_offload as g; print('GPU:', g())"
```

Al llegar a la sala, antes de levantar el modelo

Primero el audio. Es lo que más falla y lo más rápido de diagnosticar.

```
python -m hacu.voz      # ¿qué micrófono y qué altavoz hay?
python -m hacu.voz --probar      # graba 3 s y los reproduce
python -m hacu.voz --hablar "Hola, soy Hacu. Bienvenido a AudacIA."
```

En una portátil con webcam, base de conexiones y auriculares llega a haber cinco entradas. Si el sistema no elige la correcta, se fija por índice:

```
$env:HACU_ENTRADA="3"
$env:HACU_SALIDA="5"
```

Si el arranque avisa de que el motor de síntesis es el lento (piper-externo), hay pausas de segundos en cada punto:

```
python -m hacu.voz --descargar
```

Arrancar

```
python run_hacu.py --ui --voz --pantalla-completa
```

El arranque tarda: carga el modelo en la VRAM, sincroniza el corpus y precalienta. Termina en [OK] Sistema listo, HACU saluda y queda a la espera.

Si algo no arranca, hay tres escalones de respaldo, cada uno más pequeño:

Problema	Respaldo
Falla el micrófono	<code>python run_hacu.py --ui --voz-salida</code> — HACU habla, el operador escribe
Falla todo el audio	<code>python run_hacu.py --ui</code> — solo texto, la ventana funciona igual
Falla la ventana	<code>python run_hacu.py</code> — la consola hace lo mismo
Falla la GPU	Arranca en CPU: un turno tarda minutos. No sirve; mejor cancelar la demostración que enseñar eso

Durante la jornada

Entre visitante y visitante

Nuevo visitante en el panel (F9 si está oculto). Cierra el perfil, limpia la pantalla y vuelve a saludar. Sin eso, el siguiente hereda el nombre y los hechos del anterior.

Si el visitante habla por micrófono, HACU lo hace solo: nota el cambio de timbre y cierra el perfil sin que nadie pulse nada.

Ajustar al público

El desplegable **Audiencia** cambia el registro sin cambiar el contenido: *General* por defecto, *Infantil* con un colegio, *Técnico* con profesores o ingenieros, *Artístico* con gente de humanidades.

Modo trivia convierte a HACU en el que pregunta. Funciona muy bien con grupos de niños y con colas.

Cuando se pone pesado

Esc o **Callar a HACU** corta la frase de inmediato. También corta volver a pulsar la barra espaciadora: el visitante que interrumpe no tiene que esperar.

Cuando dice algo raro

HACU está construido para admitir que no sabe algo, y las pruebas lo verifican. Si aun así afirma algo que no está en el corpus, es un fallo que merece anotarse: la pregunta exacta y lo que respondió. Queda en `logs/hacu.log` con marca de tiempo.

Lo que **no** hay que hacer es discutir con él delante del público. **Nuevo visitante** y seguir.

Si se queda colgado

Ctrl+Q en la ventana, o Ctrl+C en la terminal. Cierra bien: drena la cola de memoria, cierra la base y suelta la VRAM. Volver a arrancar tarda menos que la primera vez —el corpus ya está indexado— pero sigue siendo un minuto largo.

Al cerrar

La transcripción de las conversaciones se borra sola al arrancar cuando supera las `retencion_horas` (24 por defecto). Los hechos de cada perfil se conservan.

Si la política del centro es **no conservar nada** de la jornada:

```
python -m herramientas.limpiar --aplicar --memoria
```

O desde el panel, **Borrar TODO**.

Ver *Datos de los visitantes* antes de decidir. Es una decisión de la universidad, no del operador.

Chuleta

Situación	Tecla o mando
Hablarle	Mantener Espacio
Callarlo	Esc
Ocultar el panel del operador	F9
Entrar o salir de pantalla completa	F11
Siguiente visitante	Botón Nuevo visitante
Cerrar	Ctrl+Q
Ver lo que recuerda de alguien	Botón Ver lo que recuerda
Borrar todo	Botón Borrar TODO

Datos de los visitantes

HACU guarda información de las personas que hablan con él. Esta página dice qué, dónde, cuánto tiempo y quién decide.

Qué se guarda

Dato	Dónde	Cuánto dura
Transcripción reciente de la conversación	<code>hacu_memory.db</code>	Se borra al arrancar pasadas <code>retencion_horas</code> (24 por defecto)
Hechos extraídos («se llama Daniela», «estudia ingeniería»)	<code>hacu_memory.db</code>	Hasta que alguien los borre
Registro de ejecución con las preguntas	<code>logs/hacu.log</code>	Hasta que alguien lo borre
Referencia de timbre de voz	Solo en memoria RAM	Se borra al cambiar de visitante o al cerrar

`hacu_memory.db` y `logs/` están en `.gitignore` y **no se versionan nunca**.

Qué NO se guarda

- **Ninguna huella de voz en disco.** El detector de cambio de hablante compara el timbre de lo que se acaba de decir con el de la frase anterior, en memoria, y lo olvida al pasar al siguiente visitante. No dice «esta es Camila», solo «esta es otra voz».

La distinción es lo que separa este sistema de uno biométrico. Un vector de voz almacenado es un dato biométrico, y esto es una exhibición abierta al público con menores entre el público. Guardarlo sería una decisión de la universidad, no de un commit.

- **Nada sale de la máquina.** No hay llamadas a ninguna API: el modelo, el índice y la memoria son locales.

Qué filtra el sistema por su cuenta

`hacu/sanitizer.py` decide qué merece guardarse como hecho. Descarta lo que no es información sobre el visitante: afirmaciones sobre terceros, intentos de inyección («a partir de ahora te llamas

Pepito»), meta-texto del modelo y fragmentos de sus propios prompts. `pruebas/test_unidades.py` lo verifica en el bloque sanitizer.

`identity.py` valida los nombres antes de crear un perfil: no todo lo que suena a nombre lo es, y un reconocimiento defectuoso puede producir cualquier cosa.

Cómo se borra

Alcance	Consola	Ventana	Línea de órdenes
Chat inmediato del perfil activo	2	Limpiar la pantalla	—
Memoria del perfil activo	6	—	—
Toda la base	7	Borrar TODO	<code>python -m herramientas.limpiar --aplicar --memoria</code>

El borrado de toda la base pide confirmación y no se puede deshacer.

Lo que hay que decidir antes de una exhibición real

Esto es una implementación, no una política. Antes de abrir al público, alguien de la universidad tiene que decidir:

1. **Si se conservan los hechos entre jornadas.** Ahora sí: es lo que permite que HACU reconozca a alguien que vuelve. Si la respuesta es no, hay que purgar al cerrar cada día.
2. **Cuántas horas se conserva la transcripción.** 24 por defecto (`HACU_RETENCION`).
3. **Si se informa a los visitantes** de que la conversación se recuerda durante la visita, y cómo: un cartel junto al montaje es lo habitual.
4. **Qué pasa con los menores.** Parte del público de una exhibición universitaria son niños de colegio, y el tratamiento de sus datos tiene reglas propias (Ley 1581 de 2012 y su decreto reglamentario).
5. **Si el log se conserva.** Registra las preguntas de los visitantes con marca de tiempo. Es útil para depurar y es un registro de conversaciones.

La opción más conservadora, y la que no necesita consultar a nadie, es purgar al cerrar:

```
python -m herramientas.limpiar --aplicar --memoria
```